

Loops & Iteration — একই কাজ বারবার করার Smart উপায়

JavaScript-এ অনেক সময় আমাদের একই কাজ **বারবার** করতে হয়। যেমন:

- 1 থেকে 100 পর্যন্ত number print করা
- 100 জন student-এর নাম দেখানো
- একটি shopping cart-এর সব product বের করা
- database/API থেকে পাওয়া সব data process করা
- 1, 2, 3, 4... এভাবে counting করা
- কোনো condition পূরণ না হওয়া পর্যন্ত কাজ চালানো

প্রতিবার আলাদা করে code লিখলে:

JAVASCRIPT

WITHOUT LOOP (INEFFICIENT)

```
console.log(1);  
console.log(2);  
console.log(3);  
console.log(4);  
console.log(5);
```

এভাবে 100 বার লিখতে হবে! 😞 তার পরিবর্তে **Loop** ব্যবহার করি।

PART 01 — CORE 'FOR' LOOP & NUMERIC ITERATION (5.1 – 5.12)

5.1 What is a Loop?

Loop হলো এমন একটি programming structure, যার মাধ্যমে কোনো code নির্দিষ্ট condition অনুযায়ী **বারবার execute** করা যায়।

সহজভাবে

```
একবার code লিখবে  
↓  
বারবার execute হবে  
↓  
Condition false হলে থামবে
```

REAL-LIFE EXAMPLE

ধরো teacher বললেন:

Teacher: "Class-এর 50 জন student-এর নাম একে একে বলো।"

তুমি যদি প্রতিটি student-এর জন্য আলাদা instruction দাও, অনেক সময় লাগবে।

Programming-এ:

```
Student 1  
Student 2  
Student 3  
...  
Student 50
```

এটা loop দিয়ে করা যায়।

5.2 JavaScript-এর প্রধান Loop

JavaScript-এ গুরুত্বপূর্ণ loopগুলো হলো:

Loop	ব্যবহার
<code>for</code>	নির্দিষ্ট সংখ্যক বার কাজ
<code>while</code>	condition true থাকা পর্যন্ত
<code>do ... while</code>	অন্তত একবার execute করে
<code>for ... of</code>	iterable-এর values নিয়ে কাজ
<code>for ... in</code>	object-এর keys/properties নিয়ে কাজ
Nested loop	loop-এর ভিতরে loop

এছাড়া:

JAVASCRIPT

EXAMPLE

```
break
continue
```

দিয়ে loop-এর flow control করা যায়।

5.3 `for` Loop

সবচেয়ে common loop হলো `for` loop।

SYNTAX

JAVASCRIPT

SYNTAX

```
for (initialization; condition; update) {
  // code
}
```

এখানে তিনটি গুরুত্বপূর্ণ অংশ:

```
initialization
  ↓
condition
  ↓
code
  ↓
update
  ↓
condition
  ↓
...
```

5.4 Basic for Loop

JAVASCRIPT

EXAMPLE

```
for (let i = 1; i ≤ 5; i++) {  
  console.log(i);  
}
```

OUTPUT

```
1 2 3 4 5
```

এখন একদম step-by-step দেখি।

Step 1

JAVASCRIPT

STEP 1

```
let i = 1;
```

i এর value = 1

Step 2

JAVASCRIPT

STEP 2

```
i ≤ 5
```

```
1 ≤ 5 → true
```

তাই:

JAVASCRIPT

STEP 2

```
console.log(i);
```

Output:

```
1
```

Step 3

JAVASCRIPT

STEP 3

```
i++
```

তাই:

```
i = 2
```

আবার condition check।

```
2 ≤ 5 → true
```

তারপর:

```
2
```

এভাবে চলতে থাকে।

শেষে:

```
6 ≤ 5 → false
```

তখন loop বন্ধ হয়ে যায়।

5.5 for Loop-এর তিনটি অংশ

এই code:

JAVASCRIPT

EXAMPLE

```
for (let i = 1; i ≤ 5; i++) {  
  console.log(i);  
}
```

এখানে:

1. Initialization

JAVASCRIPT

1. INITIALIZATION

```
let i = 1
```

Loop শুরু হওয়ার আগে একবার execute হয়।

2. Condition

JAVASCRIPT

2. CONDITION

```
i ≤ 5
```

প্রতিবার check হয়।

3. Update

JAVASCRIPT

3. UPDATE

```
i++
```

প্রতিবার loop body শেষ হওয়ার পর execute হয়।

5.6 Reverse Loop

শুধু সামনে নয়, পিছন দিকেও loop চালানো যায়।

JAVASCRIPT

EXAMPLE

```
for (let i = 5; i ≥ 1; i--) {  
  console.log(i);  
}
```

OUTPUT

```
5 4 3 2 1
```

এখানে:

JAVASCRIPT

EXAMPLE

```
i--
```

প্রতিবার value 1 করে কমাচ্ছে।

5.7 Even Numbers

1 থেকে 10 পর্যন্ত even number বের করি।

JAVASCRIPT

EXAMPLE

```
for (let i = 1; i ≤ 10; i++) {  
  if (i % 2 === 0) {  
    console.log(i);  
  }  
}
```

OUTPUT

2 4 6 8 10

এখানে:

JAVASCRIPT

EXAMPLE

```
i % 2 === 0
```

মানে number 2 দিয়ে ভাগ করলে remainder 0।

5.8 Odd Numbers

JAVASCRIPT

EXAMPLE

```
for (let i = 1; i ≤ 10; i++) {  
  if (i % 2 !== 0) {  
    console.log(i);  
  }  
}
```

OUTPUT

1 3 5 7 9

5.9 Sum of Numbers

1 থেকে 5 পর্যন্ত সব number-এর যোগফল বের করি।

JAVASCRIPT

EXAMPLE

```
let sum = 0;  
  
for (let i = 1; i ≤ 5; i++) {  
  sum = sum + i;  
}  
  
console.log(sum);
```

OUTPUT

15

কারণ:

0 + 1 = 1 1 + 2 = 3 3 + 3 = 6 6 + 4 = 10 10 + 5 = 15

5.10 sum += i

উপরের code আরও short করা যায়:

JAVASCRIPT

EXAMPLE

```
let sum = 0;

for (let i = 1; i ≤ 5; i++) {
  sum += i;
}

console.log(sum);
```

Output:

15

কারণ:

JAVASCRIPT

EXAMPLE

```
sum += i;
```

মানে:

JAVASCRIPT

EXAMPLE

```
sum = sum + i;
```

5.11 Multiplication Table

ধরো 5-এর নামতা।

JAVASCRIPT

EXAMPLE

```
let number = 5;

for (let i = 1; i ≤ 10; i++) {
  console.log(number + " x " + i + " = " + number * i);
}
```

OUTPUT

5 x 1 = 5 5 x 2 = 10 5 x 3 = 15 5 x 4 = 20 5 x 5 = 25 5 x 6 = 30 5 x 7 = 35 5 x 8 = 40 5 x 9 = 45 5 x 10 = 50

5.12 Template Literal দিয়ে আরও সুন্দরভাবে

JAVASCRIPT

EXAMPLE

```
let number = 5;

for (let i = 1; i ≤ 10; i++) {
  console.log(`${number} x ${i} = ${number * i}`);
}
```

Output একই:

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

এটা real-world JavaScript-এ অনেক বেশি readable।

5.13 `while` Loop

`while` loop ব্যবহার করা হয় যখন:

Teacher: "Condition true থাকা পর্যন্ত কাজ চালাও।"

SYNTAX

JAVASCRIPT

SYNTAX

```
while (condition) {
  // code
}
```

EXAMPLE

JAVASCRIPT

EXAMPLE

```
let i = 1;

while (i ≤ 5) {
  console.log(i);
  i++;
}
```

OUTPUT

```
1 2 3 4 5
```

5.14 `while` Loop কীভাবে কাজ করে?

JAVASCRIPT

EXAMPLE

```
let i = 1;
```

তারপর:

JAVASCRIPT

EXAMPLE

```
while (i ≤ 5)
```

condition check করে।

```
1 ≤ 5 → true
```

তারপর `1` print হয়।

এরপর:

JAVASCRIPT

EXAMPLE

```
i++;
```

এখন:

```
i = 2
```

আবার condition।

এভাবে:

```
1 → 2 → 3 → 4 → 5
```

শেষে:

```
6 ≤ 5 → false
```

Loop বন্ধ।

5.15 for বনাম while

for

যখন কতবার loop চলবে সেটা মোটামুটি জানা থাকে:

JAVASCRIPT

FOR

```
for (let i = 1; i ≤ 10; i++) {  
  console.log(i);  
}
```

while

যখন condition-based repetition বেশি natural:

JAVASCRIPT

WHILE

```
while (userHasNotFinished) {  
  // কাজ  
}
```

সহজভাবে:

Situation	Loop
নির্দিষ্ট counting	for
Known number of iterations	for
Condition-based	while
Unknown number of iterations	while

5.16 do ... while Loop

do ... while একটু আলাদা।

এখানে **প্রথমে code execute হবে**, তারপর condition check হবে।

SYNTAX

JAVASCRIPT

SYNTAX

```
do {  
  // code  
} while (condition);
```

EXAMPLE

JAVASCRIPT

EXAMPLE

```
let i = 1;  
  
do {  
  console.log(i);  
  i++;  
} while (i ≤ 5);
```

OUTPUT

```
1 2 3 4 5
```

5.17 while বনাম do ... while

এটা খুব important!

while

JAVASCRIPT

WHILE

```
let i = 10;

while (i ≤ 5) {
  console.log(i);
}
```

OUTPUT

কিছুই output হবে না

কারণ প্রথমেই:

```
10 <= 5 → false
```

do ... while

JAVASCRIPT

DO...WHILE

```
let i = 10;

do {
  console.log(i);
} while (i ≤ 5);
```

OUTPUT

10

কারণ `do` block প্রথমে একবার execute হয়েছে।

মনে রাখার নিয়ম

```
while
→ আগে condition
→ তারপর code

do...while
→ আগে code
→ তারপর condition
```

5.18 Real-Life Example — Login Attempt

ধরো user-কে অন্তত একবার login attempt করতে হবে।

JAVASCRIPT

EXAMPLE

```
let attempt = 1;

do {
  console.log(`Login attempt: ${attempt}`);
  attempt++;
} while (attempt ≤ 3);
```

OUTPUT

Login attempt: 1 Login attempt: 2 Login attempt: 3

5.19 Infinite Loop

যে loop কখনো শেষ হয় না তাকে **Infinite Loop** বলে।

EXAMPLE

JAVASCRIPT

EXAMPLE

```
let i = 1;

while (i ≤ 5) {
  console.log(i);
}
```

⚠️ এখানে problem হলো:

JAVASCRIPT

EXAMPLE

```
i++
```

দেওয়া হয়নি।

তাই `i` সবসময় `1` থাকবে।

Condition:

```
1 <= 5
```

সবসময় true।

ফলে loop চলতেই থাকবে।

5.20 Infinite for Loop

এটাও সম্ভব:

JAVASCRIPT

EXAMPLE

```
for (;;) {
  console.log("Hello");
}
```

এটি infinite loop।

⚠️ Browser/Node.js-এ এমন code accidentalভাবে চালালে সমস্যা হতে পারে।

5.21 break

break ব্যবহার করে loop সম্পূর্ণভাবে বন্ধ করা যায়।

EXAMPLE

JAVASCRIPT

EXAMPLE

```
for (let i = 1; i ≤ 10; i++) {  
  if (i === 5) {  
    break;  
  }  
  console.log(i);  
}
```

OUTPUT

1 2 3 4

যখন:

JAVASCRIPT

EXAMPLE

```
i === 5
```

তখন:

JAVASCRIPT

EXAMPLE

```
break;
```

loop সম্পূর্ণ stop করে দেয়।

5.22 Real-Life Example of break

ধরো 100 জন student-এর মধ্যে তুমি "Rahim" -কে খুঁজছো।

Rahim পাওয়া গেলে আর search করার দরকার নেই।

JAVASCRIPT

EXAMPLE

```
let students = ["Karim", "Sakib", "Rahim", "Hasan", "Nabil"];  
for (let i = 0; i < students.length; i++) {  
  if (students[i] === "Rahim") {  
    console.log("Rahim found!");  
    break;  
  }  
  console.log("Checking:", students[i]);  
}
```

OUTPUT

Checking: Karim Checking: Sakib Rahim found!

Rahim পাওয়ার পর loop stop হয়েছে।

5.23 continue

continue loop বন্ধ করে না।

বরং বর্তমান iteration skip করে next iteration-এ চলে যায়।

EXAMPLE

JAVASCRIPT

EXAMPLE

```
for (let i = 1; i ≤ 5; i++) {  
  if (i === 3) {  
    continue;  
  }  
  console.log(i);  
}
```

OUTPUT

1 2 4 5

3 skip হয়েছে।

5.24 break বনাম continue

এটা অবশ্যই মনে রাখবে।

break

Loop সম্পূর্ণ বন্ধ

continue

Current iteration skip
Loop চলতে থাকে

Keyword	কাজ
break	পুরো loop stop
continue	current iteration skip

5.25 for ... of

এটি modern JavaScript-এর খুব important loop।

এটি iterable-এর **values** নিয়ে কাজ করে।

Array-এর ক্ষেত্রে খুব usefull।

EXAMPLE

JAVASCRIPT

EXAMPLE

```
let fruits = ["Apple", "Banana", "Mango"];

for (let fruit of fruits) {
  console.log(fruit);
}
```

OUTPUT

```
Apple Banana Mango
```

এখানে:

JAVASCRIPT

EXAMPLE

```
fruit
```

প্রতিবার একটি value পাচ্ছে।

```
Apple
↓
Banana
↓
Mango
```

5.26 for ... of কেন useful?

Traditional for :

JAVASCRIPT

EXAMPLE

```
let fruits = ["Apple", "Banana", "Mango"];

for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

for ... of :

JAVASCRIPT

EXAMPLE

```
for (let fruit of fruits) {
  console.log(fruit);
}
```

দ্বিতীয়টা অনেক cleaner!

5.27 for ... of with String

String-ও iterable।

JAVASCRIPT

EXAMPLE

```
let name = "Shariar";

for (let character of name) {
  console.log(character);
}
```

OUTPUT

```
S h a r i a r
```

5.28 for ... of with Array

JAVASCRIPT

EXAMPLE

```
let numbers = [10, 20, 30, 40];

for (let number of numbers) {
  console.log(number);
}
```

OUTPUT

```
10 20 30 40
```

5.29 for ... in

for ... in সাধারণত Object-এর **keys/properties** iterate করতে ব্যবহার করা হয়।

EXAMPLE

JAVASCRIPT

EXAMPLE

```
let user = {
  name: "Shariar",
  age: 23,
  country: "Bangladesh"
};

for (let key in user) {
  console.log(key);
}
```

OUTPUT

```
name age country
```

এখানে:

JAVASCRIPT

EXAMPLE

```
key
```

হচ্ছে:

```
name age country
```

5.30 for ... in দিয়ে Value পাওয়া

Object-এর value পেতে:

JAVASCRIPT

EXAMPLE

```
let user = {  
  name: "Shariar",  
  age: 23,  
  country: "Bangladesh"  
};  
  
for (let key in user) {  
  console.log(key, user[key]);  
}
```

OUTPUT

```
name Shariar age 23 country Bangladesh
```

এখানে:

JAVASCRIPT

EXAMPLE

```
user[key]
```

খুব important!

যদি:

JAVASCRIPT

EXAMPLE

```
key = "name"
```

তাহলে:

JAVASCRIPT

EXAMPLE

```
user["name"]
```

→ "Shariar"

5.31 for ... in বনাম for ... of

এটা beginnerদের সবচেয়ে common confusion-এর একটি।

for ... in

→ keys

JAVASCRIPT

FOR...IN

```
let user = {
  name: "Shariar",
  age: 23
};

for (let key in user) {
  console.log(key);
}
```

Output:

```
name
age
```

for ... of

→ values

JAVASCRIPT

FOR...OF

```
let fruits = ["Apple", "Mango"];

for (let fruit of fruits) {
  console.log(fruit);
}
```

Output:

```
Apple
Mango
```

মনে রাখার নিয়ম

```
IN → index/key-এর মধ্যে
OF → value-এর মধ্যে
```

5.32 Array-তে for ... in ব্যবহার করা উচিত?

Technically করা যায়:

JAVASCRIPT

EXAMPLE

```
let fruits = ["Apple", "Banana", "Mango"];

for (let index in fruits) {
  console.log(index);
}
```

Output:

```
0
1
2
```

কিন্তু Array-এর values iterate করার জন্য সাধারণত:

JAVASCRIPT

EXAMPLE

```
for ... of
```

বা array methods বেশি appropriate।

এটা পরে **Arrays chapter**-এ বিস্তারিত দেখব।

5.33 Nested Loop

একটি loop-এর ভিতরে আরেকটি loop থাকলে তাকে **Nested Loop** বলে।

EXAMPLE

JAVASCRIPT

EXAMPLE

```
for (let i = 1; i ≤ 3; i++) {
  for (let j = 1; j ≤ 3; j++) {
    console.log(i, j);
  }
}
```

OUTPUT

```
1 1 1 1 2 1 3 2 1 2 2 2 3 3 1 3 2 3 3
```

কীভাবে কাজ করছে?

প্রথম outer loop:

```
i = 1
```

তখন inner loop পুরোটা চলে:

```
j = 1
j = 2
j = 3
```

তারপর:

```
i = 2
```

আবার inner loop পুরোটা চলে।

5.34 Nested Loop — Real-Life Example

ধরো একটি university-তে:

```
3 Departments
প্রতিটি Department-এ 3 Students
```

তাহলে:

JAVASCRIPT

EXAMPLE

```
for (let department = 1; department ≤ 3; department++) {
  for (let student = 1; student ≤ 3; student++) {
    console.log(
      `Department ${department}, Student ${student}`
    );
  }
}
```

OUTPUT

```
Department 1, Student 1 Department 1, Student 2 Department 1, Student 3 Department 2, Student 1
Department 2, Student 2 Department 2, Student 3 Department 3, Student 1 Department 3, Student 2
Department 3, Student 3
```

5.35 Pattern Printing

Nested loop বুঝতে pattern printing খুব useful।

EXAMPLE

```
*  
**  
***  
****  
*****
```

Code:

JAVASCRIPT

EXAMPLE

```
for (let i = 1; i ≤ 5; i++) {  
  
    let pattern = "";  
  
    for (let j = 1; j ≤ i; j++) {  
        pattern += "*";  
    }  
  
    console.log(pattern);  
}
```

OUTPUT

```
*  
**  
***  
****  
*****
```

5.36 Reverse Pattern

```
*****  
****  
***  
**  
*
```

Code:

JAVASCRIPT

EXAMPLE

```
for (let i = 5; i ≥ 1; i--) {  
  
    let pattern = "";  
  
    for (let j = 1; j ≤ i; j++) {  
        pattern += "*";  
    }  
  
    console.log(pattern);  
}
```

OUTPUT

```
*****  
****  
***  
**  
*
```

5.37 Loop Through Array

এটা web development-এর জন্য অত্যন্ত important।

JAVASCRIPT

EXAMPLE

```
let students = [
  "Shariar",
  "Rahim",
  "Karim",
  "Hasan"
];

for (let student of students) {
  console.log(student);
}
```

OUTPUT

Shariar Rahim Karim Hasan

Real-world application:

```
API
↓
Students Array
↓
Loop
↓
UI-তে Student Cards
```

React-এ পরে তুমি প্রায় একই ধরনের concept ব্যবহার করবে।

5.38 Array of Objects

বাস্তব application-এ data সাধারণত এমন হয়:

JAVASCRIPT

EXAMPLE

```
let students = [
  {
    name: "Shariar",
    age: 23
  },
  {
    name: "Rahim",
    age: 22
  },
  {
    name: "Karim",
    age: 24
  }
];
```

Loop:

JAVASCRIPT

EXAMPLE

```
for (let student of students) {
  console.log(student.name);
}
```

OUTPUT

Shariar Rahim Karim

আর:

JAVASCRIPT

EXAMPLE

```
for (let student of students) {
  console.log(
    `${student.name} is ${student.age} years old.`
  );
}
```

OUTPUT

Shariar is 23 years old. Rahim is 22 years old. Karim is 24 years old.

এটাই পরে API data render করার basic ideal

5.39 Loop দিয়ে Search

JAVASCRIPT

EXAMPLE

```
let products = [
  "Laptop",
  "Mouse",
  "Keyboard",
  "Monitor"
];

let search = "Keyboard";

for (let product of products) {

  if (product === search) {
    console.log("Product found!");
    break;
  }

}
```

OUTPUT

Product found!

5.40 Loop দিয়ে Maximum Number

JAVASCRIPT

EXAMPLE

```
let numbers = [10, 45, 23, 78, 12];

let max = numbers[0];

for (let number of numbers) {

  if (number > max) {
    max = number;
  }

}

console.log(max);
```

OUTPUT

78

LOGIC TRACE

```
max = 10

45 > 10 → max = 45
23 > 45 → No
78 > 45 → max = 78
12 > 78 → No
```

শেষে:

78

5.41 Loop দিয়ে Minimum Number

JAVASCRIPT

EXAMPLE

```
let numbers = [10, 45, 23, 78, 12];

let min = numbers[0];

for (let number of numbers) {

    if (number < min) {
        min = number;
    }

}

console.log(min);
```

OUTPUT

10

5.42 Loop দিয়ে Count

ধরো array-তে কয়টি even number আছে।

JAVASCRIPT

EXAMPLE

```
let numbers = [1, 2, 4, 7, 8, 10, 13];

let count = 0;

for (let number of numbers) {

    if (number % 2 === 0) {
        count++;
    }

}

console.log(count);
```

OUTPUT

4

কারণ even numbers:

2, 4, 8, 10

মোট:

4

5.43 break + continue একসাথে

JAVASCRIPT

EXAMPLE

```
for (let i = 1; i ≤ 10; i++) {  
  if (i === 3) {  
    continue;  
  }  
  
  if (i === 8) {  
    break;  
  }  
  
  console.log(i);  
}
```

OUTPUT

1 2 4 5 6 7

কী হয়েছে?

3 → continue → skip
8 → break → stop

PART 06 — SCOPE, ADVANCED MECHANICS & PRACTICAL MINI-PROJECTS (5.44 – 5.52)

5.44 Loop-এর Scope

let দিয়ে loop variable তৈরি করলে সেটি সাধারণত loop block-এর মধ্যে সীমাবদ্ধ থাকে।

JAVASCRIPT

EXAMPLE

```
for (let i = 1; i ≤ 3; i++) {  
  console.log(i);  
}
```

এর পরে:

JAVASCRIPT

EXAMPLE

```
console.log(i);
```

করলে error হবে:

ReferenceError

কারণ `i` loop-এর বাইরে accessible নয়।

5.45 `var` এবং Loop

পুরনো JavaScript-এ:

JAVASCRIPT

EXAMPLE

```
for (var i = 1; i ≤ 3; i++) {  
  console.log(i);  
}  
  
console.log(i);
```

এখানে output:

```
1  
2  
3  
4
```

কারণ `var` function-scoped।

এই কারণে modern JavaScript-এ সাধারণত loop variable-এর জন্য:

JAVASCRIPT

EXAMPLE

```
let
```

ব্যবহার করা হয়।

5.46 Loop-এর `const`

`for ... of` -এ `const` খুব common:

JAVASCRIPT

EXAMPLE

```
let fruits = ["Apple", "Mango", "Banana"];  
  
for (const fruit of fruits) {  
  console.log(fruit);  
}
```

এখানে প্রতিটি iteration-এ নতুন `fruit` binding পাওয়া যায়।

5.47 Labelled Statement — Advanced Concept

JavaScript-এ loop-কে label দেওয়া যায়।

JAVASCRIPT

EXAMPLE

```
outerLoop:
for (let i = 1; i ≤ 3; i++) {

  for (let j = 1; j ≤ 3; j++) {

    if (i === 2 && j === 2) {
      break outerLoop;
    }

    console.log(i, j);
  }
}
```

OUTPUT

```
1 1 1 2 1 3 2 1
```

এখানে:

JAVASCRIPT

EXAMPLE

```
break outerLoop;
```

শুধু inner loop নয়, পুরো outer loop-ও stop করেছে।

⚠️ এটি valid JavaScript, তবে normal code-এ খুব কম ব্যবহার করা হয়।

5.48 Which Loop Should You Use?

একটা practical guide:

নির্দিষ্ট সংখ্যক iteration:

JAVASCRIPT

নির্দিষ্ট সংখ্যক ITERATION:

for

Example:

JAVASCRIPT

নির্দিষ্ট সংখ্যক ITERATION:

```
for (let i = 0; i < 10; i++) {}
```

Condition-based:

JAVASCRIPT

CONDITION-BASED:

while

Example:

JAVASCRIPT

CONDITION-BASED:

```
while (balance > 0) {}
```

অন্তত একবার execute করতে হবে:

JAVASCRIPT

অন্তত একবার EXECUTE করতে হবে:

do ... while

Array-এর values:

JAVASCRIPT

ARRAY-এর VALUES:

for ... of

Object-এর keys:

JAVASCRIPT

OBJECT-এর KEYS:

for ... in

5.49 Loop vs Array Methods

Modern JavaScript-এ শুধু loop নয়, array methods-ও খুব গুরুত্বপূর্ণ।

যেমন:

JAVASCRIPT

EXAMPLE

```
map()  
filter()  
reduce()  
find()  
some()  
every()  
forEach()
```

এগুলো আমরা **Arrays Chapter**-এ বিস্তারিতভাবে শিখব।

উদাহরণ:

JAVASCRIPT

EXAMPLE

```
let numbers = [1, 2, 3];  
  
numbers.forEach(function(number) {  
  console.log(number);  
});
```

Output:

```
1  
2  
3
```

এখন শুধু জানবে:

Teacher: `forEach()` একটি array method, traditional loop statement নয়।

5.50 Mini Project — Student Result System

এবার Chapter 4 + Chapter 5-এর concept একসাথে ব্যবহার করি।

JAVASCRIPT

EXAMPLE

```
let students = [
  {
    name: "Shariar",
    marks: 85
  },
  {
    name: "Rahim",
    marks: 72
  },
  {
    name: "Karim",
    marks: 58
  },
  {
    name: "Hasan",
    marks: 31
  }
];

for (let student of students) {

  let grade;

  if (student.marks ≥ 80) {
    grade = "A+";
  } else if (student.marks ≥ 70) {
    grade = "A";
  } else if (student.marks ≥ 60) {
    grade = "A-";
  } else if (student.marks ≥ 50) {
    grade = "B";
  } else if (student.marks ≥ 40) {
    grade = "C";
  } else {
    grade = "F";
  }

  console.log(
    `${student.name} → ${student.marks} → ${grade}`
  );
}
```

OUTPUT

```
Shariar → 85 → A+
Rahim → 72 → A
Karim → 58 → B
Hasan → 31 → F
```

🔥 এখানে তুমি একসাথে ব্যবহার করেছো:

Array Object for ... of if else if else Template Literal

পরবর্তী chapter-গুলোতে এগুলো আরও powerful হবে।

5.51 Mini Project — Shopping Cart Total

JAVASCRIPT

EXAMPLE

```
let cart = [
  {
    name: "Keyboard",
    price: 1500
  },
  {
    name: "Mouse",
    price: 800
  },
  {
    name: "Headphone",
    price: 2000
  }
];

let total = 0;

for (let product of cart) {
  total += product.price;
}

console.log("Total:", total);
```

OUTPUT

Total: 4300

LOGIC TRACE

```
1500
+ 800
+ 2000
-----
4300
```

এটাহি basic shopping cart-এর একটা core logic।

5.52 Mini Project — Find Product

JAVASCRIPT

EXAMPLE

```
let products = [
  "Laptop",
  "Keyboard",
  "Mouse",
  "Monitor",
  "Headphone"
];

let search = "Monitor";
let found = false;

for (let product of products) {

  if (product === search) {
    found = true;
    break;
  }
}

if (found) {
  console.log("Product found.");
} else {
  console.log("Product not found.");
}
```

OUTPUT

Product found.

PART 07 — COMPREHENSIVE CHEAT SHEET, COMPARISONS & PRACTICE SET

CHEAT SHEET 🗨 Chapter 5 Cheat Sheet

```
FOR
→ নির্দিষ্টভাবে loop চালাতে

WHILE
→ condition true থাকা পর্যন্ত

DO...WHILE
→ অন্তত একবার চালাতে

FOR...OF
→ values পেতে

FOR...IN
→ keys পেতে

BREAK
→ পুরো loop stop

CONTINUE
→ current iteration skip

NESTED LOOP
→ loop-এর ভিতরে loop
```

COMPARISON 🔥 সবচেয়ে গুরুত্বপূর্ণ পার্থক্য (Key Loop Differences)

for

JAVASCRIPT

FOR

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

while

JAVASCRIPT

WHILE

```
let i = 0;  
  
while (i < 5) {  
  console.log(i);  
  i++;  
}
```

do ... while

JAVASCRIPT

DO...WHILE

```
let i = 0;  
  
do {  
  console.log(i);  
  i++;  
} while (i < 5);
```

for ... of

JAVASCRIPT

FOR...OF

```
for (let value of array) {  
  console.log(value);  
}
```

for ... in

JAVASCRIPT

FOR...IN

```
for (let key in object) {  
  console.log(key);  
}
```



Basic Questions

1. Loop কী?
2. `for` loop-এর তিনটি অংশ কী?
3. `while` এবং `do ... while` -এর পার্থক্য কী?
4. `break` কী করে?
5. `continue` কী করে?
6. `for ... of` কী return করে?
7. `for ... in` কী return করে?
8. Infinite loop কী?
9. Nested loop কী?
10. `for ... of` এবং `for ... in` -এর পার্থক্য কী?

 Coding Practice## Coding Practice

Problem 1 — 1 to 100

1 থেকে 100 পর্যন্ত print করো।

Problem 2 — Even Numbers

1 থেকে 50 পর্যন্ত সব even number print করো।

Problem 3 — Odd Numbers

1 থেকে 50 পর্যন্ত সব odd number print করো।

Problem 4 — Sum

1 থেকে 100 পর্যন্ত সব number-এর sum বের করো।

Expected:

5050

Problem 5 — Multiplication Table

যেকোনো একটি number-এর 1 থেকে 10 পর্যন্ত multiplication table তৈরি করো।

Problem 6 — Maximum

JAVASCRIPT

EXAMPLE

```
let numbers = [10, 55, 23, 89, 12, 67];
```

সবচেয়ে বড় number বের করো।

Expected:

89

Problem 7 — Minimum

একই array থেকে সবচেয়ে ছোট number বের করো।

Expected:

10

Problem 8 — Search

JAVASCRIPT

EXAMPLE

```
let students = [  
  "Shariar",  
  "Rahim",  
  "Karim",  
  "Hasan",  
  "Nabil"  
];
```

Karim আছে কিনা check করো।

Problem 9 — Count Even

JAVASCRIPT

EXAMPLE

```
let numbers = [1, 2, 4, 7, 8, 11, 14, 20];
```

কয়টি even number আছে সেটা বের করো।

Problem 10 — Pattern

এই pattern তৈরি করো:

```
*  
**  
***  
****  
*****
```

তারপর:

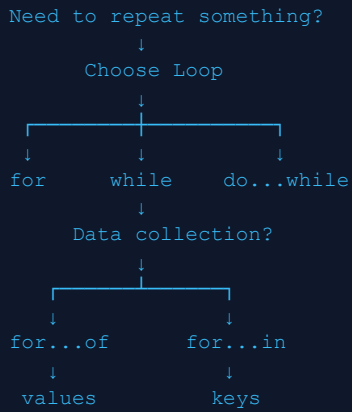
```
*****  
****  
***  
**  
*
```

দুটোই নিজে code করার চেষ্টা করো।

SUMMARY

🚀 Chapter 5-এর মূল শিক্ষা

এখন তোমার মাথায় এই flow-টা clear থাকা উচিত:



আর সবচেয়ে important:

```
break
→ Loop শেষ

continue
→ এই iteration বাদ

Nested Loop
→ Loop-এর ভিতরে Loop
```

NEXT CHAPTER

🚀 Chapter 6 Preview — Functions

পরের chapter-এ আমরা যাব **Chapter 6 — Functions**-এ। সেখানে JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ building blocks-এর একটি নিয়ে বিস্তারিত করব: **Function কী, parameter, argument, return, function declaration/expression, arrow function, default parameter, rest parameter, callback function, higher-order function, scope, এবং practical functions।**